Vigilant: LLM and RAG-Enhanced Agentic Framework for Continuous Software Traceability and Consistency Assurance across Requirements, Code, and Tests

Ankith Singh

***Abstract*—Software changes can invalidate relationships among requirements, implementations, and tests even when individual artifacts remain syntactically valid. Trace-link recovery alone does not establish that related artifacts remain consistent. Vigilant combines versioned artifact extraction, a structural knowledge graph, hybrid retrieval, and a bounded agent workflow to assemble verifiable evidence for change investigations. A local Qwen2.5-1.5B-Instruct model produces constrained decisions, while deterministic rules and model judgments remain separately recorded. We evaluate trace-link ranking on SMOS, iTrust, English eTour, and a module-level RTMX subset, totaling 251 linked queries, and examine 12 authored change scenarios through 60 model calls. Compared with hybrid retrieval, RAG-assisted LLM reranking changes mean average precision from 0.3026 to 0.3061 on SMOS and from 0.4406 to 0.4427 on eTour, but reduces it on iTrust and RTMX. All consistency variants return uncertain decisions, yielding 25% agreement with authored labels, equal to an always-uncertain baseline. A recorded repository investigation verifies 24 evidence excerpts but does not establish semantic correctness. Consistency detection remains unresolved. The contribution is an auditable implementation and empirical separation of ranking, evidence provenance, impact coverage, and semantic correctness.**

***Index Terms*—software traceability, requirements engineering, retrieval-augmented generation, knowledge graphs, large language models, change impact analysis, agentic workflows**

I. Introduction

Requirements, code, and tests evolve at different rates. A renamed endpoint may leave stale client calls; a revised requirement may leave implementation and tests unchanged. Traceability identifies artifact relationships, whereas consistency assessment asks whether related artifacts agree at a particular revision. A plausible trace link is evidence for investigation, not proof of conformance.

Lexical retrieval, learned representations, and language models support trace-link recovery [1], but similarity and static connectivity alone do not establish conformance. RAG supplies external evidence to a generator [2]; LiSSA already applies it to multiple traceability tasks [3]. We investigate how revision-specific evidence and bounded tools support review while preserving uncertainty about semantic correctness.

Vigilant implements a repository analysis workflow in which artifacts, graph relationships, retrieved excerpts, rule findings, and model outputs retain distinct identities and provenance. Its agent roles execute a prescribed sequence of tools; they do not form an autonomous population of independently reasoning models. Similarly, "continuous" denotes the implemented ability to trigger analysis on repository changes, not an experimentally established guarantee of continuous correctness.

This paper contributes a version-aware implementation and an auditable empirical separation of retrieval, evidence provenance, impact coverage, and consistency decisions. Four retrieval datasets, 12 controlled changes, and a recorded investigation expose mixed ranking effects and unresolved consistency detection. No new retrieval algorithm, novelty priority, or superior model is claimed.

II. Related Work

Retrieval and learned traceability. The review by Alturayeif *et al.* organizes machine-learning approaches to software traceability [1]. BM25 provides a lexical ranking basis [4], whereas Sentence-BERT provides a framework for sentence embeddings suitable for similarity retrieval [5]. These methods motivate complementary baselines rather than an assumption that dense representations dominate lexical matching. Vigilant uses a pretrained MiniLM encoder and retains per-project comparisons to expose differences in vocabulary and link density.

RAG, structure, and decision scope. LiSSA evaluates generic link recovery across artifact types [3]; Hey *et al.* study inter-requirements RAG [6]; and Fuchß *et al.* investigate LLM candidate filtering [7]. Vigilant's ranking experiment only reorders five retrieved requirement–code candidates. Its separate change workflow attaches Git revisions, checked source spans, structural neighbors, rule hypotheses, and an explicit consistency decision to a review record. Thus, ranking evaluation and revision-specific consistency evaluation have different units and labels; published scores are not treated as comparable baselines.

Architecture-documentation inconsistency detection [8] and TransArC's architecture-mediated documentation–code links [9] establish relevant structural precedents. Vigilant uses source-level calls, imports, tests, and literal API relations to propose revision-local impacts, without asserting that these paths prove conformance. SWE-agent studies agent–computer interfaces [10]; Vigilant instead executes a fixed tool sequence.

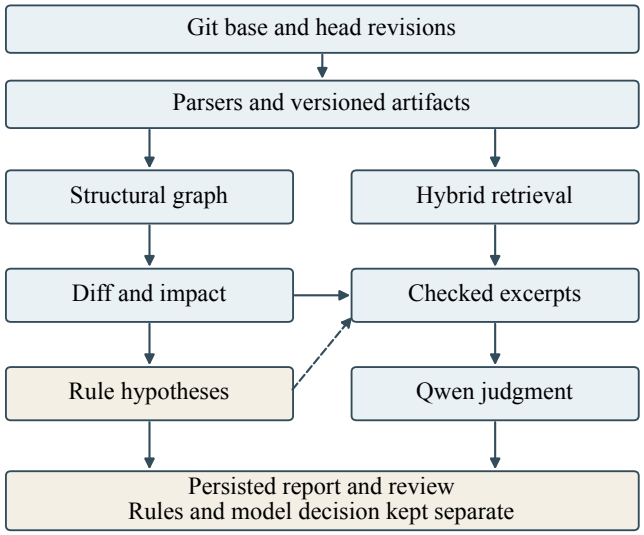


Fig. 1. Implemented investigation architecture. A prescribed workflow orchestrates the components. Rule findings and model decisions are persisted separately. The dashed arrow adds rule-targeted evidence; relationships remain candidates.

The contribution is this implemented separation of evidence and decisions, assessed with negative results, rather than integration alone as a novelty claim.

III. Proposed Methodology and System Architecture

A. Versioned Artifacts and Structural Graph

The implementation uses a Python/FastAPI backend, SQLAlchemy persistence with SQLite, and a React interface. Git base and head revisions supply repository snapshots. Python AST parsing and Tree-sitter parsers for Java, JavaScript, and TypeScript extract supported source artifacts; requirement text, tests, and literal API declarations or client calls provide additional evidence. Imported repository code is parsed, not executed. Unsupported constructs and dynamic dispatch limit extraction completeness.

Each canonical artifact records project, repository, path, type, symbol, revision, and content hash. Identity and content version are separate: a stable symbol can change content between commits. NetworkX represents a directed multigraph whose persisted relationships include calls, imports, tests, API exposure and consumption, dependencies, and commit modifications. Static name matching and literal routes support these edges; they are not a whole-program semantic analysis. Recovered similarity links retain candidate status instead of becoming trusted implementation relationships.

Figure 1 shows the investigation data flow. Base and head graphs are composed for impact exploration. The implementation traverses an undirected view for at most two hops, excluding default reverse-containment expansion. The traversal proposes candidates from either revision; reachability does not confirm a defect.

B. Retrieval and Evidence Assembly

Lexical retrieval uses normalized tokens, including split identifiers, with TF-IDF or BM25. Dense retrieval uses `all-MiniLM-L6-v2`, producing 384-dimensional vectors [11]. Target text is divided into chunks of 180 normalized words, with at most eight chunks. Chunk vectors are averaged and normalized. Embeddings use CPU float32, batch size 24, four threads, and a model/text-keyed cache. Chunk caps can omit late file content.

Hybrid retrieval applies reciprocal rank fusion (RRF) [12]:

$$S(q,d) = \frac{1}{60 + r_B(q,d)} + \frac{1}{60 + r_E(q,d)}, \tag{1}$$

where r_B and r_E are one-based BM25 and embedding ranks of target d for query q. The graph baseline combines 0.85 times normalized hybrid relevance with 0.15 times mean neighboring relevance from extracted file-level call, import, and test edges. Isolated targets retain their own relevance as the neighborhood term. Gold trace links are not used as graph edges. These exploratory weights are fixed for the reported run, not established optimal parameters.

Two RAG paths must be distinguished. The investigation path combines retrieved and graph-selected artifacts, changed seeds, and available before/after evidence. Retrieval and graph selection each use a depth of eight; an initial 8,000-character budget collects excerpts of up to 24 lines, with additional targeted temporal and rule evidence. Thus, the initial budget is not a strict bound on the final assembled packet. Each excerpt is checked against its source revision, path, lines, and hash. The measured investigation prompt then selects only the first six excerpts and at most 400 characters from each; the question is limited to 1,200 characters. Checked evidence in the report can therefore exceed what the model actually sees.

The traceability experiment instead presents a requirement prefix of at most 900 characters and five candidate prefixes of at most 1,100 characters each. Qwen labels candidates `SUPPORTED`, `UNSUPPORTED`, or `UNCERTAIN` using evidence aliases. Retrieval supplies evidence; the generated labels only control reordering. Supported candidates move ahead within the original top five; remaining order is stable. The fixed top-five and top-ten sets make Recall@5/@10 invariant. We call this operation RAG-assisted LLM reranking, distinct from agentic investigation.

C. LLM, Agent Workflow, and Decisions

The measured generator is `Qwen2.5-1.5B-Instruct` [13], executed locally on CPU in bfloat16 with four threads. Decoding is greedy, inputs exceeding 3,072 tokens are rejected, output at 220 new tokens, and a grammar constrains JSON generation. Neither the generator nor encoder is fine-tuned. Consistency labels must agree across the decision fields. A non-uncertain judgment requires positive confidence and a supplied evidence citation; unknown evidence/artifact aliases cause an uncertain fallback. These syntactic conditions do not establish entailment. `SUPPORTED/UNSUPPORTED` belong to the separate trace-candidate schema, not the consistency schema.

TABLE I
Datasets and principal experimental settings

Dataset / role	Req.	Linked	Targets	Links
SMOS / train	67	67	100	1,044
iTrust / validation	131	105	226	286
eTour English / test	58	57	116	308
RTMX modules / external	22	22	7	22

Component	Configuration
Encoder	MiniLM-L6-v2; 384 dimensions; 180 words/chunk; at most 8 chunks
Generator	Qwen2.5-1.5B-Instruct; greedy; CPU bfloat16; 220 output tokens
Retrieval	TF-IDF, BM25, cosine, RRF ($k = 60$); graph weight 0.15
Reranking	First 5 hybrid candidates; supported-first, stable order
Consistency	12 authored scenarios; 5 variants; 60 calls

The 12 prescribed stages resolve revisions, import snapshots, compute differences, build base and head graphs, collect impact candidates, recover trace candidates, retrieve evidence, apply rules, verify excerpts, request a model decision, and persist the run. Role labels organize responsibilities; the model does not choose tools autonomously. Rules flag removed endpoints, changed code with unchanged tests, deleted last-known tests, or obsolete client routes. These findings remain review hypotheses, stored separately from model status, confidence, citations, and uncertainty. Tool completion does not imply a resolved investigation.

A repository watcher can trigger analysis when Git HEAD changes. The implementation uses in-process execution and SQLite rather than a durable distributed job queue. An OmniRoute adapter was subsequently implemented, but no live gateway evaluation is included; every reported LLM result below comes from the pinned local Qwen model.

IV. Experimental Setup

A. Research Questions and Data

The evaluation asks: RQ1, how do ranking methods compare under the same linked-query policy? RQ2, do additional evidence and bounded investigation improve consistency judgments on controlled changes? RQ3, can a complete change investigation retain checked evidence and distinguish execution success from model certainty?

Table I summarizes the retrieval datasets and configuration. SMOS, iTrust, and English eTour originate from the LiSSA replication package, pinned to commit `a8e652f29bbd`; RTMX [14] is pinned to `c2ced5349577`. Full commit hashes and model revisions accompany the reproduction materials. The three Java datasets serve as training, validation, and test projects for the optional learned ranker, respectively; the non-learned baselines share the same prepared targets and queries. RTMX is an external, module-level subset, not a function-level benchmark. Two missing modules and unresolved function-level annotations preclude interpreting its 22 retained queries as the original broader task.

B. Comparisons, Metrics, and Runtime

The retrieval evaluation includes TF-IDF, BM25, MiniLM, hybrid RRF, hybrid plus structural graph relevance, and hybrid plus local-LLM reranking. The graph experiment covers the three Java datasets; RTMX has no reported graph run. A class-balanced logistic ranker uses TF-IDF, normalized BM25, embedding similarity, and Jaccard features, with random state 42 and a 1,000-iteration limit. Its retention rule requires validation MAP to exceed the strongest retrieval baseline; test performance does not determine retention.

Only requirements with at least one annotated link contribute to ranking metrics. This excludes 26 iTrust and one eTour requirements and does not evaluate no-link detection. For linked query set Q, with relevant target set G_q, average precision and MAP are

$$\begin{aligned} \mathrm{AP}(q) &= \frac{\sum_{k=1}^{N_q} P_q(k)\ \mathrm{rel}_q(k)}{|G_q|}, \\ \mathrm{MAP} &= \frac{1}{|Q|} \sum_{q \in Q} \mathrm{AP}(q). \end{aligned} \tag{2}$$

where N_q is the candidate count, $P_q(k)$ is precision through rank k, and $\mathrm{rel}_q(k)$ indicates an annotated link. The saved evaluation also reports MRR, nDCG@10, mean per-query Recall@K, and micro precision, recall, and F1 for the top-ten decisions. Micro recall and mean per-query Recall@10 have different denominators and are not interchangeable.

The consistency configuration study uses 12 authored Git mutations across RTMX, Flask, and a full-stack FastAPI/React repository with expected labels: six potential inconsistencies, three uncertain cases, two broken traces, and one missing-test case. Five configurations each make 12 actual calls: question only; hybrid evidence; hybrid plus graph; additional bounded inspection; and temporal evidence plus rules. This is a configuration comparison, not an isolated ablation: inputs and budgets differ, and graph-assisted contexts have authored baseline trace relations. Affected-artifact oracles are incomplete; outside-oracle endpoints are unjudged. Saved rules-only findings are reported separately in the supplement, without forcing them into a single-label accuracy metric.

The resumed consistency-run record identifies an AMD EPYC 9V74 host and an 8 GiB container memory limit; it does not establish exclusive use of the host. Recorded runs use Python 3.12 on Linux, PyTorch 2.7.1 CPU, Transformers 4.53.2, and Sentence-Transformers 5.0.0. Results represent the saved configurations, with resumable calls, rather than repeated randomized trials. A post-hoc paired bootstrap resamples per-query AP differences within each dataset (20,000 replicates; seeds 20261015–20261018), yielding percentile 95% intervals conditional on the frozen model and prompt. These assume exchangeable queries, may understate correlation, and are not project-generalization intervals or multiplicity-adjusted tests. Independent arithmetic recomputation within the project audit checks 23 method rows and 1,484 query rows; a separate observation audit checks all 251

TABLE II
MAP on linked queries (higher is better)

Method	SMOS	iTrust	eTour	RTMX
TF-IDF	0.2924	0.3160	0.4102	0.7894
BM25	0.2979	0.3787	0.3074	0.7606
MiniLM	0.2859	0.2061	0.4492	0.6795
Hybrid	0.3026	0.3068	0.4406	0.7515
Hybrid + KG	0.2965	0.2958	0.4744	—
Hybrid + RAG	0.3061	0.3043	0.4427	0.6985

RTMX denotes the module-valid subset. A dash indicates NOT RUN. These are per-dataset descriptive comparisons, without significance claims.

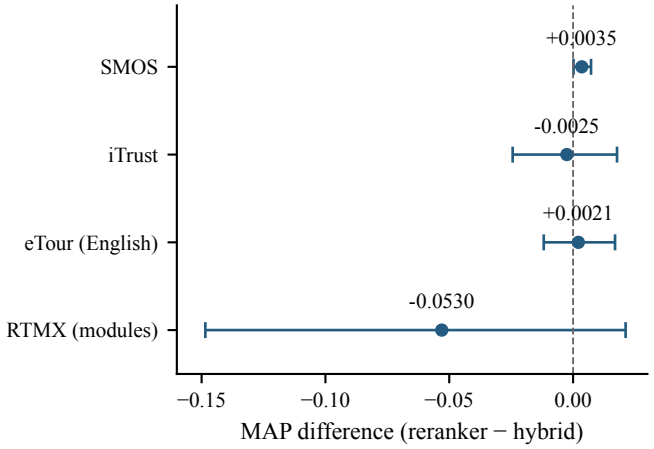


Fig. 2. MAP differences produced by LLM reranking. Points show signed changes; bars show descriptive paired-query bootstrap 95% intervals. Positive values favor reranking.

reranking calls. These checks validate record consistency, not independent human judgments.

V. Results and Analysis

A. Trace-Link Ranking

Table II reports MAP under the shared query policy. No method dominates across datasets. BM25 is strongest on iTrust (0.3787), and TF-IDF is strongest on the seven-module RTMX subset (0.7894). On eTour, MiniLM alone (0.4492) exceeds hybrid retrieval (0.4406); adding lexical evidence is therefore not uniformly beneficial. Hybrid plus graph reaches 0.4744 on eTour, but falls below hybrid on SMOS and iTrust.

RAG-assisted LLM reranking produces small MAP increases on SMOS (+0.0035) and eTour (+0.0021), alongside decreases on iTrust (-0.0025) and RTMX (-0.0530), as shown in Fig. 2. Rankings change for 17/67, 60/105, 33/57, and 19/22 queries, respectively. Thus, frequent changes do not imply improved ranking. The 95% intervals are $[0.0002, 0.0073]$, $[-0.0244, 0.0177]$, $[-0.0119, 0.0169]$, and $[-0.1485, 0.0212]$. Only SMOS excludes zero under this query-level resampling; no cross-project superiority follows. All 251 responses pass the recorded full-schema checks, yet this structural success coexists with lower retrieval effectiveness on two datasets. Semantic entailment and hallucination rates were not independently annotated.

As expected from the reranking rule, Recall@5 and Recall@10 remain unchanged. RTMX Recall@10 equals one for every method because its seven targets all fit within that cutoff. This ceiling makes MAP and early-rank behavior more informative than top-ten recall there. Likewise, eTour's graph-assisted MAP improvement does not imply improvement in every metric: micro top-ten precision, recall, and F1 change from 0.2930, 0.5422, and 0.3804 under hybrid retrieval to 0.2860, 0.5292, and 0.3713 with the graph. The learned ranker is rejected: validation MAP is 0.3051, below BM25's 0.3787.

B. Consistency and Change Investigation

Every configuration predicts `UNCERTAIN` in 12/12 cases, matching the always-uncertain baseline: 25% authored-label agreement, macro-F1 0.10, and 0% non-uncertain coverage. For each variant, the confusion matrix has only its UNCERTAIN column nonzero: 6 potential inconsistencies, 2 broken traces, 1 missing test, and 3 uncertain cases. UNCERTAIN precision/recall/F1 are 0.25/1.00/0.40; the other classes have zero recall and F1. Their undefined precision is assigned zero for macro averaging. Matrices and per-class metrics are provided separately; repeated configurations are not 60 independent cases.

All 60 raw outputs already select UNCERTAIN with confidence zero; five additionally fail citation-alias validation. The validator therefore does not explain the universal abstention. The prompt includes an uncertain example, and 46 outputs give "Evidence is incomplete" as their entire reason. Prompt anchoring and restricted excerpts are plausible explanations, not isolated causal findings. Citation validity is 12/12, 11/12, 11/12, 11/12, and 10/12 across configurations. Consistency detection remains unresolved.

Recorded known-endpoint recall increases across configurations from 0 to 0, 0.20, 0.32, and 0.72. This indicates improved coverage of the partial authored endpoint sets in the richer configurations, not independently verified impact accuracy. In particular, temporal evidence and rule hypotheses are introduced together. Their individual effects cannot be separated. The broader controlled change records also retain a consumer-change miss involving four known endpoints, contradicting any perfect-impact claim.

The end-to-end example changes an API route from `/items` to `/catalog` while a client retains the old route. The saved run completes 12 tool stages, identifies 11 changed artifact records and 33 impact candidates, and verifies 24 source excerpts. Fifteen rule findings comprise ten lost-endpoint and five obsolete-consumer findings associated with the shared route change; they are not 15 independently confirmed defects. The model again returns uncertain. Total recorded duration is 33.65 seconds, including 25.10 seconds for the model stage. Observed mean generation durations for the four reranking datasets range from 37.93 to 70.82 seconds per query; these are observations under the saved CPU runs, not a controlled throughput benchmark.

VI. Discussion

RQ1 is dataset-dependent: lexical baselines remain competitive, graph relevance changes ranking trade-offs, and a constrained generation stage can worsen retrieved order. The bootstrap describes variation across the observed queries, not across models, prompts, or future projects. Repeating deterministic calls would not supply that missing variation.

For RQ2, evidence coverage rises while semantic label agreement remains at the abstention baseline. Rules-only outputs are multi-label hypothesis sets: their inclusion of authored labels is not a comparable single-label accuracy result or independent correctness evidence. A controlled follow-up must fix cases, prompt budget, model, and decision policy while varying one evidence source, with independently adjudicated labels. The accompanying protocol specifies these controls without claiming an unrun experiment.

For RQ3, three outcomes remain distinct: evidence validity checks a quote against a revision; impact coverage measures recovery of known endpoints; semantic correctness requires the conclusion to follow from evidence. The pipeline demonstrates the first and partial coverage, not the third. Continuous operation is an implemented change trigger, with no longitudinal validation. Local execution avoids per-call API charges but consumes hardware and time; the unevaluated OmniRoute extension establishes neither accuracy nor universally free operation.

VII. Limitations and Threats to Validity

Construct validity. Ranking quality is not requirements conformance. Source-quote verification establishes provenance, not entailment; schema validity establishes output structure, not truth. Authored consistency labels and incomplete endpoint sets limit the meaning of agreement and coverage. The benchmark excludes unlinked queries, so its results cannot characterize false-positive behavior for requirements with no implementation.

Internal and conclusion validity. There is one saved configuration per method and no independent semantic annotation. Bootstrap intervals are post-hoc and conditional on correlated within-project queries; no confirmatory significance claim is made. Exploratory graph weights and prompt choices may affect outcomes. The consistency configurations change several factors together, preventing an isolated attribution to graph use, agents, or temporal evidence. Reported latency includes differing contexts and resumable execution conditions. Audit agreement confirms arithmetic and record provenance, not the correctness of benchmark labels.

External validity. Three Java retrieval datasets and a small Python module subset do not represent industrial polyglot systems. Controlled mutations are not independently discovered production defects. Static parsing misses dynamic behavior, chunk and prompt caps discard content, and the small local model is only one point in the quality–resource trade-off. The watcher has not been evaluated longitudinally; durable distributed execution and live OmniRoute accuracy remain unvalidated. These limits preclude claims of universal continuous assurance or general model superiority.

VIII. Conclusion and Future Work

Vigilant implements version-aware extraction, structural graphs, hybrid retrieval, checked evidence, and bounded agent orchestration for investigating relationships among requirements, code, and tests. Across 251 linked queries, local RAG-assisted LLM reranking yields mixed MAP changes; the 60-call consistency study remains entirely uncertain and matches a 25% always-uncertain baseline. A complete investigation demonstrates traceable evidence collection, while retaining uncertainty about semantic correctness. Future work should obtain independent artifact and consistency annotations, include no-link queries, isolate component effects, evaluate multiple model and prompt configurations under fixed budgets, and measure longitudinal behavior on real repository changes. Consistency assurance is an objective, not a validated capability. The present evidence supports an auditable review framework and a reproducible evaluation.

References

[1] N. Alturayeif, J. Hassine, and I. Ahmad, "Machine learning approaches for automated software traceability: A systematic literature review," *J. Syst. Softw.*, vol. 230, Art. no. 112536, 2025, doi: 10.1016/j.jss.2025.112536.

[2] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 9459–9474.

[3] D. Fuchß *et al.*, "LiSSA: Toward generic traceability link recovery through retrieval-augmented generation," in *Proc. ICSE*, 2025, pp. 1396–1408, doi: 10.1109/ICSE55347.2025.00186.

[4] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009, doi: 10.1561/1500000019.

[5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992, doi: 10.18653/v1/D19-1410.

[6] T. Hey, D. Fuchß, J. Keim, and A. Koziolek, "Requirements traceability link recovery via retrieval-augmented generation," in *Proc. REFSQ*, LNCS 15588, 2025, pp. 381–397, doi: 10.1007/978-3-031-88531-0_27.

[7] D. Fuchß, S. Schwedt, J. Keim, and T. Hey, "Beyond retrieval: A study of using LLM ensembles for candidate filtering in requirements traceability," in *Proc. IEEE REW*, 2025, pp. 5–12, doi: 10.1109/REW66121.2025.00006.

[8] J. Keim, S. Corallo, D. Fuchß, and A. Koziolek, "Detecting inconsistencies in software architecture documentation using traceability link recovery," in *Proc. ICSA*, 2023, pp. 141–152, doi: 10.1109/ICSA56044.2023.00021.

[9] J. Keim, S. Corallo, D. Fuchß, T. Hey, T. Telge, and A. Koziolek, "Recovering trace links between software documentation and code," in *Proc. ICSE*, 2024, Art. no. 215, doi: 10.1145/3597503.3639130.

[10] J. Yang *et al.*, "SWE-agent: Agent-computer interfaces enable automated software engineering," arXiv:2405.15793, 2024.

[11] Sentence Transformers, "all-MiniLM-L6-v2," model card, revision 1110a243fdf4, accessed Oct. 15, 2026. [Online]. Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[12] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. SIGIR*, 2009, pp. 758–759, doi: 10.1145/1571941.1572114.

[13] Qwen Team, "Qwen2.5-1.5B-Instruct," model card, revision 989aa7980e4c, accessed Oct. 15, 2026. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct

[14] RTMX AI, "rtmx-bench," source repository, commit c2ced5349577, accessed Oct. 15, 2026. [Online]. Available: https://github.com/rtmx-ai/rtmx-bench